

ANDROID · TEST TASK

شرح المهمة وخطة التنفيذ

Authentication & Secure Token Handling

دليل كامل: ما المطلوب، كيف يسير الكود، العمليات المعقدة مشروحة بالتفصيل، خطة التنفيذ خطوة بخطوة، وقاموس المصطلحات الإنجليزية.

Kotlin

MVVM

Retrofit

OkHttp

Interceptor

Token Refresh

Concurrency

RecyclerView

المصدر: FakeStore API · Android_Test_Task.pdf

1. ملخص المهمة في جملة

2. ال APIs المستخدمة (FakeStore)

3. المتطلبات الوظيفية بالتفصيل (Functional Requirements)

4. المعمارية المقترحة (Architecture)

5. سير الكود خطوة بخطوة (Code Flow)

6. قرارات تصميم مهمة (ناقشها في README وفي المقابلة)

7. خطة التنفيذ (للمرحلة القادمة)

8. كيف تُثبت أن المهمة تعمل؟ (Testing)

9. قاموس المصطلحات (English → عربي)

10. الأخطاء الشائعة التي يجب تجنبها ⚠️

11. أسئلة متوقعة في المقابلة

12. ال Skills والأدوات المفيدة مع Claude Code لاحقاً

13. قائمة التحقق النهائية قبل التسليم ✅

ملحق: العمليات المعقدة مشروحة بالتفصيل

1. ملخص المهمة في جملة

بناء تطبيق Android بسيط يسجل دخول المستخدم، ويحفظ الـ token بأمان، ويرفقه بكل طلب، ويجدده تلقائياً عند انتهائه، مع ضمان أن عدة طلبات متزامنة لا تسبب أكثر من طلب تجديد واحد.

💡 **جوهر المهمة ليس الشاشات.** هو منطق الـ **Token: Interceptor**، وانتهاء الصلاحية، والتجديد، والتزامن. هذا ما سيختبره المدير في المقابلة.

2. ال APIs المستخدمة (FakeStore)

الطلب	الوظيفة	ملاحظات
POST <code>https://fakestoreapi.com/auth/login</code>	تسجيل الدخول	Body: <code>{"username": "mor_2314", "password": "83r5^_"}</code> والرد: <code>{"token": "..."} </code>
GET <code>https://fakestoreapi.com/products</code>	قائمة المنتجات	تُعامل ك محمية
GET <code>https://fakestoreapi.com/products/{id}</code>	تفاصيل منتج	تُعامل ك محمية

⚠ **نقطة مهمة جداً:** سيرفر FakeStore **لا يتحقق فعلياً** من ال token في طلبات المنتجات، أي أنها ستعمل حتى بدونه. **لكن المطلوب أن تتصرف كأنه يتحقق:**

- ترفق `Authorization: Bearer <token>` بكل طلب منتجات.
- تكتشف انتهاء ال token **بنفسك داخل التطبيق** (بعد 60 ثانية)، لأن السيرفر لن يخبرك أبداً بالرد `401`.

3. المتطلبات الوظيفية بالتفصيل (Functional Requirements)

3.1 شاشة تسجيل الدخول (Login Screen)

المطلوب	المعنى العملي
حقل Username و Password	EditText (أو TextInputLayout من Material)
زر Login	يرسل الطلب عبر ال ViewModel
حالة Loading	إظهار ProgressBar وتعطيل الزر أثناء الطلب
حالة Error	رسالة واضحة: بيانات خاطئة، لا يوجد إنترنت، خطأ سيرفر
حفظ ال token بأمان	ليس SharedPreferences العادي، بل تخزين مشفر (انظر القسم 6)

3.2 التعامل مع ال Token (المتطلب الأساسي ★)

أ) Authorization Interceptor كود يعمل تلقائياً على كل طلب قبل إرساله، ويضيف ال header:

```
Authorization: Bearer eyJhbGciOi...
```

فلا تكتب ال header يدوياً في كل استدعاء.

ب) محاكاة انتهاء الصلاحية (Token Expiry Simulation)

- صلاحية ال token = 60 ثانية.
- عند حفظ ال token تحفظ معه وقت الحفظ tokenSavedAt.
- قبل أي طلب: isExpired = now - tokenSavedAt >= 60_000ms.

ج) سلوك ال token المنتهي (Expired Token Behavior)

- ممنوع إرسال طلب ب token منتهٍ.
- إذا كان منتهياً، جددّه أولاً تلقائياً ثم أرسل الطلب بال token الجديد، دون أن يشعر المستخدم بشيء.
- د) التزامن (Concurrency Handling)، أصعب جزء السيناريو: ال token انتهى، وفي اللحظة نفسها انطلقت 3 طلبات (مثلاً القائمة وتفاصيل منتج).

- ❌ الخطأ: كل طلب يكتشف الانتهاء ويطلق تجديداً خاصاً به، فتُرسل 3 طلبات login.
- ✅ الصحيح: طلب تجديد واحد فقط، والطلبان الآخران ينتظران ثم يكملان بال token الجديد.

3.3 تدفق التجديد المُحاكى (Refresh Token Flow)

- FakeStore لا يملك endpoint للتجديد، فالتجديد = استدعاء /auth/login مرة أخرى بنفس بيانات المستخدم.
- كل هذا المنطق يوضع في كلاس واحد اسمه AuthManager (أو ما يشبهه).
- تجنّب الحلقات اللانهائية (infinite retry loops): إذا فشل التجديد فلا تعد المحاولة إلى ما لا نهاية.
- إذا فشل التجديد → تسجيل خروج إجباري (force logout).

3.4 واجهة المنتجات (Products UI)

الشاشة	التفاصيل
قائمة المنتجات	<code>RecyclerView</code> (مذكور بالاسم، فهو ليس Jetpack Compose): صورة وعنوان وسعر
تفاصيل المنتج	صورة، عنوان، سعر، وصف، تصنيف

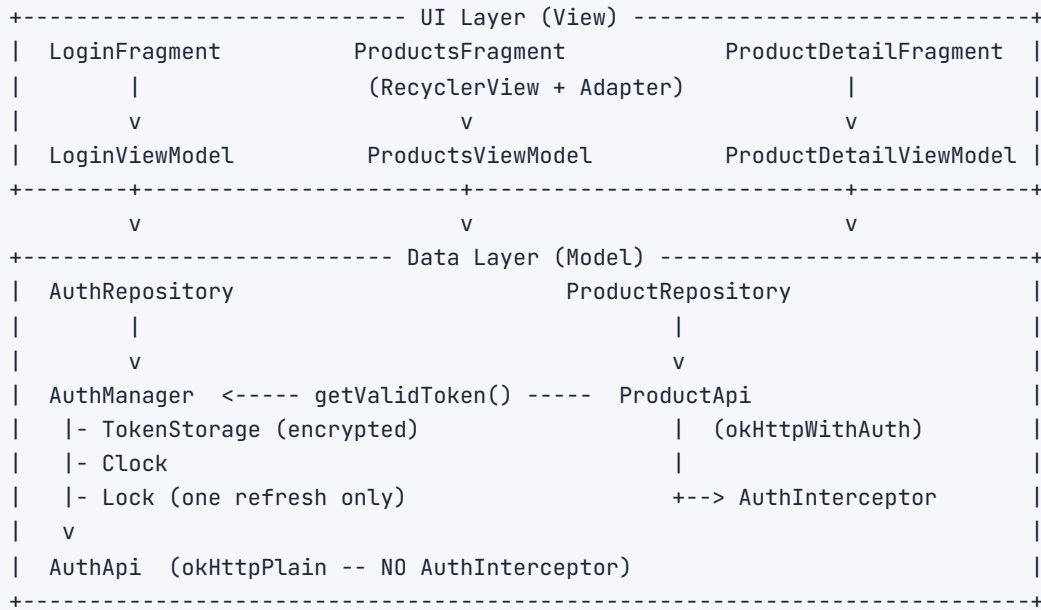
3.5 تسجيل الخروج (Logout)

- مسح ال token (وأي بيانات اعتماد محفوظة).
- العودة لشاشة Login مع مسح ال **back stack**، فلا يستطيع المستخدم الرجوع لشاشة المنتجات بزر الرجوع.

3.6 المتطلبات التقنية (Technical Requirements)

المطلوب	المعنى
MVVM	فصل الشاشة (View) عن المنطق (ViewModel) عن البيانات (Model/Repository)
Retrofit + OkHttp	Retrofit لتعريف الطلبات، وOkHttp لتنفيذها مع ال Interceptors

4. المعمارية المقترحة (Architecture)



قراءة المخطط:

- **الطبقة العليا (View):** كل شاشة لها ViewModel، والشاشة تتعامل معه فقط.
- **الطبقة السفلى (Model):** الـ ViewModels تطلب البيانات من الـ Repositories.
- **AuthManager** هو "عقل" الـ token، ويحتوي على:
- **TokenStorage**: تخزين مشفر للـ token و tokenSavedAt وبيانات الدخول.
- **Clock**: مصدر الوقت، ويمكن استبداله في الاختبارات.
- **Lock**: قفل يضمن تجديداً واحداً فقط.
- **ProductApi** يمر عبر **AuthInterceptor**، الذي يسأل **AuthManager** عن token صالح قبل كل طلب.
- **AuthApi** (login) يستخدم عميل **OkHttp بدون AuthInterceptor**.

لماذا نحتاج نسختين من OkHttpClient؟

- **ProductApi**: يمر عبر **AuthInterceptor** ليحصل على الـ token.
- **AuthApi** (login): **يجب ألا يمر عبر AuthInterceptor**. لو مرّ عبره، فالـ Interceptor سيرى أن الـ token منتهٍ، فيستدعي login، الذي يمر عبر الـ Interceptor، فيستدعي login مجدداً... وهكذا **حلقة لا نهائية (infinite recursion)**. هذا من أهم الفخاخ في المهمة.

```
com.example.authapp/  
├── data/  
│   ├── remote/      AuthApi, ProductApi, dto/ (LoginRequest, LoginResponse, Product)  
│   ├── auth/        AuthManager, TokenStorage, AuthInterceptor, TokenAuthenticator, Clock  
│   └── repository/   AuthRepository, ProductRepository  
├── di/              NetworkModule, StorageModule (Hilt)  
├── ui/  
│   ├── login/       LoginFragment, LoginViewModel, LoginUiState  
│   ├── products/    ProductsFragment, ProductsViewModel, ProductAdapter  
│   ├── detail/      ProductDetailFragment, ProductDetailViewModel  
│   └── MainActivity  (listens for ForceLogout)  
└── util/            UiState, Result
```


5. سير الكود خطوة بخطوة (Code Flow)

5.1 تسجيل الدخول

#	الخطوة	الكود المعني
1	المستخدم يكتب البيانات ويضغط Login	LoginFragment
2	الحالة تصبح Loading، فيظهر ProgressBar ويُعطل الزر	LoginViewModel.login(u, p)
3	الطلب ينتقل لطبقة البيانات	AuthManager.login(u, ← AuthRepository p)
4	إرسال الطلب للسيرفر عبر عميل بلا AuthInterceptor	AuthApi.login()
5	عند النجاح: حفظ الـ token مع وقت الحفظ وبيانات الدخول، ثم الانتقال للمنتجات ومسح Login من الـ back stack	TokenStorage.save(token, tokenSavedAt = now, credentials)
6	عند الفشل: رسالة خطأ واضحة	state = Error("Invalid username or password")

5.2 طلب منتجات بـ token صالح

#	الخطوة	الكود المعني
1	الـ ViewModel يطلب المنتجات	← ProductRepository ← ProductsViewModel.load() ProductApi.getProducts()
2	قبل الإرسال، الـ Interceptor يعترض الطلب	AuthInterceptor.intercept()
3	يسأل عن token صالح، وبما أن عمره أقل من 60 ثانية يعود فوراً	AuthManager.isValidToken()
4	يضيف الـ header ويرسل الطلب	Authorization: Bearer <token>
5	الرد يتحول لقائمة، والحالة تصبح Success، والقائمة تُعرض	adapter.submitList() ← List<Product>

5.3 طلب بعد انتهاء الـ token (بعد 60 ثانية)

#	الخطوة
1	AuthInterceptor يستدعي AuthManager.isValidToken()
2	الـ token منتهٍ ، فيدخل القفل (lock)
3	يتحقق مرة ثانية داخل القفل ، وربما جدد طلب آخر قبله

#	الخطوة
4	إذا ما زال منتهياً، يستدعي <code>AuthApi.login(credentials)</code> بالبيانات المحفوظة
5 ✓	نجاح: يحفظ token جديداً و <code>tokenSavedAt</code> جديداً، ثم يُكمل الطلب الأصلي بال token الجديد
5 ✗	فشل: <code>clear()</code> ثم إطلاق حدث <code>ForceLogout</code> ، والطلب يفشل دون إعادة محاولة

5.4 التزامن: 3 طلبات وال token منتهٍ

الطلبات A و B و C تنطلق معاً وال token منتهٍ:

اللحظة	طلب A	طلب B	طلب C
1	منتهٍ؟ نعم، يأخذ القفل 🔒	منتهٍ؟ نعم، ينتظر القفل ⌚	منتهٍ؟ نعم، ينتظر القفل ⌚
2	يرسل <code>login</code> ويحفظ token جديداً	ينتظر ⌚	ينتظر ⌚
3	يحرر القفل 🔒 ويرسل طلبه ✓	يأخذ القفل ويتحقق: صالح الآن!	ينتظر ⌚
4	—	لا <code>login</code> ، يحرر القفل ويرسل ✓	يأخذ القفل ويتحقق: صالح الآن!
5	—	—	لا <code>login</code> ، يرسل ✓

النتيجة: طلب `login` واحد فقط لثلاثة طلبات.

هذا النمط اسمه **Double-Checked Locking**: تتحقق قبل القفل، ثم تتحقق مرة أخرى بعد أخذه. التحقق الثاني هو ما يمنع B و C من إطلاق تجديد مكرر.

فكرة الكود (الفهم فقط):

```
// AuthManager
@synchronized // one lock for all requests (or ReentrantLock)
fun getValidToken(): String? {
    val current = storage.token
    if (current != null && !isExpired()) return current // second check, inside the lock
    return refresh() // only one login call
}
```

لماذا قفل عادي (`synchronized`) وليس `Mutex` من `coroutines`؟ لأن `Interceptor.intercept()` في `OkHttp` دالة متزامنة (**blocking**) تعمل على `threads` خاصة بـ `OkHttp`، وليست `suspend`. القفل العادي هو الأنسب هنا. (البديل { } { } `mutex.withLock` يعمل أيضاً، لكنه أقل وضوحاً.)

Force Logout 5.5

- التجديد يفشل، فيستدعي `AuthManager.clear()` ثم `_events.emit(ForceLogout)` عبر `SharedFlow`.
- `MainActivity` تستمع لهذه الأحداث، فتستقبل `ForceLogout`.
- تنتقل لشاشة Login مع `graph, inclusive = true` (بداية الـ `popUpTo`) لمسح كل الشاشات السابقة.

4. تعرض رسالة: "Session expired, please log in again".

5.6 Logout اليدوي

- المستخدم يضغط زر **Logout** في الـ **Toolbar**.
- يُستدعى `ProductsViewModel.logout()` ، ثم `AuthManager.clear()`.
- الانتقال لشاشة Login مع مسح الـ `back stack`.

5.7 فتح التطبيق

عند فتح التطبيق، تفحص `MainActivity` (أو شاشة Splash): هل يوجد **token** محفوظ؟

- **نعم:** تفتح شاشة المنتجات. حتى لو انتهى الـ `token`، سيُجدّد تلقائياً عند أول طلب.
- **لا:** تفتح شاشة Login.

5.8 منع الحلقات اللانهائية (Avoid infinite retry loops)

الخطر	الحل
login يمر عبر <code>AuthInterceptor</code>	<code>AuthApi</code> منفصل لـ <code>Retrofit/OkHttp</code>
التجديد يفشل ثم يُعاد للأبد	محاولة تجديد واحدة لكل طلب، وعند الفشل <code>force logout</code>
(اختياري) السيرفر يرد 401 بعد التجديد	<code>TokenAuthenticator</code> يعيد المحاولة مرة واحدة فقط (يفحص <code>response.priorResponse</code>)، ثم يُرجع <code>null</code>

6. قرارات تصميم مهمة (ناقشها في README وفي المقابلة)

6.1 أين نحفظ ال token بأمان؟

الخيار	التقييم
SharedPreferences عادي	✗ نص مقروء، مرفوض لأن المطلوب "Securely"
EncryptedSharedPreferences (مكتبة androidx.security:security-crypto)	✓ الأسهل والأشهر في المهام الاختبارية. لكن Google أوقفت تطويرها (deprecated)، فاذكر ذلك
DataStore + تشفير بمفتاح من Android Keystore (أو مكتبة Tink)	✓✓ الحل الحديث الموصى به، أطول قليلاً

التوصية: EncryptedSharedPreferences خلف interface اسمه TokenStorage، مع ملاحظة في README أنه يمكن استبداله بـ DataStore + Keystore دون تغيير بقية الكود. هذا يُظهر وعياً معمارياً.

6.2 مشكلة "التجديد يحتاج كلمة المرور"

بما أن التجديد = login جديد، نحتاج اسم المستخدم وكلمة المرور عند كل تجديد:

- **خيار أ:** حفظها مشفرة مع ال token. التجديد يعمل حتى بعد إغلاق التطبيق.
- **خيار ب:** إبقاؤها في الذاكرة فقط. أكثر أماناً، لكن بعد إعادة فتح التطبيق وانتهاء ال token يحدث force logout.

التوصية: خيار أ، مع توضيح في README أن التطبيق الحقيقي لا يحفظ كلمة المرور أبداً، بل يستخدم refresh token من السيرفر، وأن هذا مجرد محاكاة لأن FakeStore لا يوفر واحداً.

6.3 مصدر الوقت (Clock)

- System.currentTimeMillis(): مناسب لأن tokenSavedAt يُحفظ ويبقى بعد إعادة التشغيل.
- اجعله خلف interface Clock، فتستطيع في الاختبارات "تقديم الوقت" 61 ثانية دون انتظار فعلي.

6.4 فحص استباقي أم تفاعلي؟ (Proactive vs Reactive)

- **استباقي (Proactive):** ال Interceptor يفحص العمر قبل الإرسال. هذا هو المطلوب، لأن FakeStore لن يرد بـ 401 أبداً.
- **تفاعلي (Reactive):** Authenticator يتصرف بعد رد 401. إضافة احترافية اختيارية تُظهر أنك تعرف كيف تعمل التطبيقات الحقيقية.

7. خطة التنفيذ (للمرحلة القادمة)

#	المرحلة	المخرَج	التحقق
1	إعداد المشروع	مشروع Kotlin بـ Views (ليس Compose)، و ViewBinding، و Gradle dependencies، وصلاحية INTERNET	التطبيق يفتح
2	طبقة الشبكة	AuthApi، ProductApi، DTOs، نسختنا OkHttp	طلب login يعمل ويطبع Logcat في token
3	التخزين الآمن	TokenStorage مشفّر (token، tokenSavedAt، credentials)	البيانات تبقى بعد إعادة فتح التطبيق
4	★ AuthManager	login، getValidToken مع قفل، refresh، clear، حدث ForceLogout	Unit tests (القسم 8)
5	AuthInterceptor	يضيف الـ header ويستدعي getValidToken	Logcat: header موجود في كل طلب منتجات
6	شاشة Login	ViewModel + Fragment + حالات Loading/Error	بيانات خاطئة تظهر رسالة، وصحيحة تنقلك للمنتجات
7	قائمة المنتجات	RecyclerView + ListAdapter + DiffUtil + تحميل الصور (Coil/Glide)	القائمة تظهر بالصور
8	تفاصيل المنتج	تمرير id عبر Repository + ViewModel + Navigation	الضغط يفتح التفاصيل الصحيحة
9	Logout + Force Logout	زر في الـ Toolbar، والاستماع للحدث في MainActivity	الرجوع لا يعيدك للمنتجات
10	إثبات التزامن	زر/سيناريو يطلق عدة طلبات معاً بعد 60 ثانية	Logcat: طلب login واحد فقط
11	التلميع	README، تنظيف الكود، commits واضحة	مراجعة بـ /code-review

8. كيف تُثبت أن المهمة تعمل؟ (Testing)

اختبارات يدوية

1. سجّل دخولك، **انتظر 61 ثانية**، ثم افتح منتجاً. في Logcat ستجد طلب `/auth/login` تلقائياً، ثم طلب المنتج.
2. انتظر 61 ثانية، ثم اضغط Refresh في القائمة وافتح منتجاً بسرعة. يجب أن يظهر **login واحد فقط**.
3. سجّل دخولك، ثم **عدّل كلمة المرور المحفوظة** (كود debug) لتصبح خاطئة، وانتظر 61 ثانية. يجب أن يحدث force logout مع رسالة.
4. اضغط Logout، ثم زر الرجوع. يجب أن يخرج من التطبيق، لا أن يعود للمنتجات.
5. افصل الإنترنت. يجب أن تظهر رسالة خطأ و زر Retry، لا crash.

AuthManager J Unit Tests (تُبهر المراجع)

الاختبار	الفكرة
token صالح → لا تجديد	<code>FakeClock</code> عند 30s، و <code>FakeAuthApi.loginCount == 0</code>
token منتهٍ → تجديد واحد	<code>FakeClock</code> عند 61s، ثم <code>loginCount == 1</code>
10 طلبات متزامنة → تجديد واحد	10 threads تستدعي <code>getValidToken()</code> معاً، ثم <code>loginCount == 1</code>
فشل التجديد → force logout	<code>FakeAuthApi</code> يرمي exception، فتُمسح البيانات ويُطلق الحدث

9. قاموس المصطلحات (English → عربي)

المصادقة والأمان

المصطلح	المعنى
Authentication	المصادقة: التحقق من هوية المستخدم (من أنت؟)
Authorization	التفويض: ما المسموح لك به. وهو أيضاً اسم ال header الذي يحمل ال token
Token	نص طويل يعطيه السيرفر بعد الدخول، ويثبت هويتك في الطلبات التالية بدل إرسال كلمة المرور كل مرة
Bearer Token	نوع token يعني "من يحمله يُعتبر صاحبه"، ويُرسل بالشكل <code>Authorization: Bearer <token></code>
JWT	JSON Web Token: صيغة شائعة لل token (الذي يرجعه FakeStore من هذا النوع)
Access Token	ال token قصير العمر المستخدم في الطلبات
Refresh Token	token طويل العمر يُستخدم فقط للحصول على access token جديد. FakeStore لا يوفّره، لذلك نحاكيه ب login
Token Expiry / Expiration	انتهاء صلاحية ال token (هنا 60 ثانية)
tokenSavedAt	الوقت (بالميلي ثانية) الذي حُفظ فيه ال token، ويُستخدم لحساب العمر
Refresh Flow	سلسلة خطوات تجديد ال token تلقائياً
Force Logout	إخراج المستخدم إجبارياً (مثلاً عند فشل التجديد)
Credentials	بيانات الدخول: اسم المستخدم وكلمة المرور
Secure Storage	تخزين مشفّر لا يمكن قراءته حتى لو حصل أحد على ملفات التطبيق
EncryptedSharedPreferences	نسخة مشفّرة من SharedPreferences (مكتبة security-crypto)
Android Keystore	خزنة مفاتيح تشفير في نظام Android، لا يمكن استخراج المفاتيح منها
DataStore	البديل الحديث ل SharedPreferences (غير متزامن ويعتمد على Flow)

الشبكة

المصطلح	المعنى
API	واجهة يقدمها السيرفر لتطلب منه بيانات
Endpoint	عنوان محدد في ال API، مثل <code>/products/{id}</code>
HTTP Method (GET/POST)	GET لجلب بيانات، و POST لإرسال بيانات (مثل login)

المصطلح	المعنى
Request Body	البيانات المرسلّة مع POST (هنا JSON فيه username و password)
Response	رد السيرفر
Header	معلومات إضافية ترافق الطلب، مثل Authorization و Content-Type
Status Code	رقم نتيجة الطلب: 200 نجاح، 401 غير مصادّق، 404 غير موجود، 500 خطأ سيرفر
Unauthorized 401	السيرفر يرفض الـ token (منتهِ أو خاطئ). FakeStore لا يرسله للمنتجات
JSON	صيغة نصية لنقل البيانات
DTO (Data Transfer Object)	كلاس يطابق شكل JSON القادم أو المرسل، مثل LoginRequest و LoginResponse
Retrofit	مكتبة تحوّل interface في Kotlin إلى طلبات HTTP
OkHttp	المحرك الذي ينفّذ الطلبات فعلياً. Retrofit مبني فوقه
OkHttpClient	كائن الإعدادات: الـ interceptors، والمهلة الزمنية (timeouts)...
Interceptor	كود "يعترض" كل طلب قبل إرساله أو كل رد بعد وصوله، ليعدّله أو يسجّله
Authorization Interceptor	Interceptor مهمته إضافة Authorization: Bearer لكل طلب
Authenticator (OkHttp)	يُستدعى تلقائياً عند رد 401 ليحدّد الـ token ويعيد الطلب
HttpLoggingInterceptor	يطبع الطلبات والردود في Logcat، وهو ضروري لإثبات أن التجديد حدث مرة واحدة
Converter (Gson/Moshi)	يحوّل JSON إلى كائنات Kotlin والعكس
Retry	إعادة محاولة الطلب
Infinite Retry Loop	حلقة إعادة محاولة لا تنتهي: تجديد يفشل فيُعاد للأبد، أو login يستدعي نفسه

التزامن (Concurrency)

المصطلح	المعنى
Concurrency	تنفيذ عدة مهام في الوقت نفسه
Concurrent Requests	عدة طلبات شبكة تنطلق معاً
Thread	مسار تنفيذ مستقل. OkHttp ينفّذ الطلبات على threads متعددة
Main / UI Thread	الـ thread الذي يرسم الواجهة، ويُمنع تنفيذ الشبكة عليه
Race Condition	خطأ يحدث عندما تصل عدة threads لنفس البيانات معاً، فتتغير النتيجة حسب "من يصل أولاً"
Lock / synchronized	قفل يسمح لـ thread واحد فقط بدخول جزء من الكود في المرة الواحدة
Mutex	قفل خاص بالـ coroutines (kotlinx.coroutines.sync.Mutex)

المصطلح	المعنى
Double-Checked Locking	تحقق قبل القفل ثم تحقق ثاني بعد أخذه، لتجنّب عمل مكرر
Blocking	دالة توقف الـ thread حتى تنتهي (مثل <code>Interceptor.intercept</code>)
Coroutine / suspend	تنفيذ عمل طويل دون حجز الـ thread أو تجميد الواجهة
SharedFlow	قناة لبث أحداث لمرة واحدة (مثل <code>ForceLogout</code>) لكل من يستمع
StateFlow	يحمل الحالة الحالية دائماً (مثل <code>Loading/Success/Error</code>)

الواجهة و Android

المصطلح	المعنى
MVVM	Model - View - ViewModel: فصل الشاشة عن المنطق عن البيانات
View	الشاشة (Activity/Fragment + XML)
ViewModel	يحمل حالة الشاشة ومنطقها، ويبقى حياً عند تدوير الشاشة
Model / Repository	طبقة البيانات، والـ Repository هو الواجهة الوحيدة للبيانات أمام الـ ViewModel
UI State	حالة الشاشة: Loading / Success / Error
Activity	شاشة كاملة
Fragment	جزء شاشة داخل Activity. نمط شائع: Activity واحدة وعدة Fragments
XML Layout	ملف يصف شكل الشاشة (مطلوب لأن RecyclerView من نظام Views)
ViewBinding	يولّد كلاساً للوصول لعناصر XML بأمان بدل <code>findViewById</code>
RecyclerView	قائمة فعالة تعيد تدوير (recycle) عناصر العرض عند التمرير بدل إنشاء جديدة
Adapter	يربط البيانات بعناصر الـ RecyclerView
ViewHolder	يمسك عناصر عرض صف واحد لإعادة استخدامها
ListAdapter + DiffUtil	Adapter يحسب الفرق بين القائمة القديمة والجديدة، ويحدّث ما تغيّر فقط
Navigation Component	مكتبة التنقل بين Fragments عبر ملف <code>nav_graph.xml</code>
Back Stack	سجل الشاشات المفتوحة. زر الرجوع يعود للشاشة السابقة فيه
popUpTo / inclusive	مسح شاشات من الـ back stack أثناء التنقل (مطلوب في Logout)
Loading State	حالة "جاري التحميل" (ProgressBar)
Error State	حالة "حدث خطأ" (رسالة + Retry)
Logcat	نافذة السجلات في Android Studio

المصطلح	المعنى
Dependency Injection / Hilt	توفير الـ dependencies للكلاسات من الخارج بدل إنشائها بداخلها، وHilt هو أداة Google لذلك (اختياري هنا لكنه احترافي)
Coil / Glide	مكتبات لتحميل الصور من الإنترنت وعرضها

المتطلبات والتسليم

المصطلح	المعنى
Functional Requirements	ما يجب أن يفعله التطبيق (شاشات، سلوك)
Technical Requirements	كيف يجب أن يُبنى (MVVM، Retrofit، OkHttp)
Must Have	إلزامي، والتسليم ناقص بدونه
Encapsulate	تغليف المنطق داخل كلاس واحد (AuthManager) وإخفاء تفاصيله عن الباقي
Simulate	محاكاة سلوك غير موجود فعلياً (الانتهاء والتجديد)
Unit Test	اختبار جزء صغير من الكود بشكل معزول
Fake	نسخة وهمية من كلاس تُستخدم في الاختبارات (مثل FakeClock و FakeAuthApi)

10. الأخطاء الشائعة التي يجب تجنبها ⚠️

1. استدعاء login عبر نفس OkHttpClient الذي فيه AuthInterceptor ، مما يسبب حلقة لا نهائية.
2. نسيان التحقق الثاني داخل القفل، فيحدث تجديد مكرر رغم وجود القفل.
3. استخدام SharedPreferences عادي وتسميته "آمن".
4. إرسال الطلب أولاً ثم فحص الانتهاء. المطلوب منع إرسال token منته.
5. إعادة محاولة التجديد بلا حد عند الفشل بدل force logout.
6. Logout دون مسح الـ back stack، فيعود المستخدم للمنتجات بزر الرجوع.
7. استخدام Jetpack Compose بدل RecyclerView المذكور صراحة.
8. وضع منطق الـ token داخل الـ ViewModel أو الـ Fragment بدل AuthManager.
9. تنفيذ الشبكة على الـ Main thread، فيظهر crash NetworkOnMainThreadException.
10. ترك Log يطبع الـ token أو كلمة المرور في نسخة release.

11. أسئلة متوقعة في المقابلة

السؤال	جواب مختصر
ما الفرق بين <code>Authenticator</code> و <code>Interceptor</code> ؟	<code>Interceptor</code> يعمل على كل طلب (استباقي)، و <code>Authenticator</code> يعمل فقط عند 401 (تفاعلي)
كيف ضمنت تجديداً واحداً فقط؟	قفل واحد مع Double-Checked Locking داخل <code>AuthManager</code>
لماذا <code>synchronized</code> وليس <code>Mutex</code> ؟	لأن <code>intercept()</code> دالة blocking على threads الـ <code>OkHttp</code> وليست suspend
كيف تجنبنا الحلقات اللانهائية؟	<code>OkHttp</code> منفصل لـ login، ومحاولة تجديد واحدة، و force logout عند الفشل
أين حفظت الـ token ولماذا؟	تخزين مشفر بمفتاح من Android Keystore، لأن <code>SharedPreferences</code> العادي مقروء
لماذا حفظت كلمة المرور؟ وهل هذا آمن؟	لمحاكاة التجديد فقط. في الإنتاج نستخدم refresh token ولا نحفظ كلمة المرور أبداً
كيف اختبرت التزامن؟	Unit test بـ 10 threads و <code>FakeClock</code> ، والتحقق من <code>loginCount == 1</code>
ماذا لو أعاد السيرفر 401 رغم أن الـ token "صالح" عندنا؟	<code>Authenticator</code> يجدد ويعيد المحاولة مرة واحدة فقط، ثم force logout

12. ال Skills والأدوات المفيدة مع Claude Code لاحقاً

المرحلة	الأداة	لماذا
بداية المشروع	<code>/init</code>	ينشئ <code>CLAUDE.md</code> يوثق بنية المشروع وقواعده
التخطيط	Plan mode	قبل كتابة <code>AuthManager</code> (الجزء الأخطر)
بعد كل مرحلة	<code>/code-review</code>	لاكتشاف أخطاء التزامن والحلقات اللانهائية
قبل التسليم	<code>/security-review</code>	مهم جداً هنا: تخزين ال token وكلمة المرور، وعدم تسريبهما في ال logs
التنظيف	<code>/simplify</code>	تبسيط الكود قبل التسليم

13. قائمة التحقق النهائية قبل التسليم

- ☐ شاشة Login بحقلين وزر وحالتي Error Loading
- ☐ ال token محفوظ مشفراً مع tokenSavedAt
- ☐ AuthInterceptor يضيف Authorization: Bearer لكل طلبات المنتجات
- ☐ لا يُرسل أي طلب ب token عمره ≤ 60 ثانية
- ☐ التجديد التلقائي يعمل (login مخفي)
- ☐ عدة طلبات متزامنة تسبب login واحداً فقط (مُثبت ب Logcat أو unit test)
- ☐ منطق ال token كله داخل AuthManager
- ☐ login لا يمر عبر AuthInterceptor
- ☐ فشل التجديد يؤدي إلى force logout دون حلقات
- ☐ قائمة المنتجات ب RecyclerView
- ☐ شاشة تفاصيل المنتج
- ☐ Logout يمسح البيانات ويمسح ال back stack
- ☐ MVVM + Retrofit + OkHttp
- ☐ README يشرح المعمارية وقرارات التصميم (القسم 6)
- ☐ commits واضحة على GitHub

ملحق: العمليات المعقدة مشروحة بالتفصيل

أ) كيف يعمل الـ `Interceptor`؟ (سلسلة المعتراضات)

تشبيه: الطلب رسالة تمر على عدة موظفين في ممر قبل أن تخرج من المبنى، والرد يعود عبر نفس الموظفين بالترتيب العكسي.

```
ProductApi.getProducts()
|
v
[ AuthInterceptor ]      adds Authorization: Bearer <token>
|   chain.proceed(request)
|
v
[ HttpLoggingInterceptor ]  logs the request in Logcat
|
v
Network --> Server
|
^   the response travels back through the same chain (reverse order)
```

- كل `Interceptor` يستلم `chain`، ويعدّل الطلب إن أراد، ثم يستدعي `chain.proceed(newRequest)` ليمرره للتالي.
- الترتيب مهم:** نضع `AuthInterceptor` قبل `HttpLoggingInterceptor`، ليظهر الـ `header` في السجل.
- `intercept()` تعمل على **thread خلفي** خاص بـ `OkHttp`، لذلك يُسمح فيها بالانتظار (`blocking`) دون تجميد الواجهة.

ب) لماذا قفل + تحقق مزدوج؟ (Double-Checked Locking)

تشبيه: ثلاثة موظفين يجدون أن **مفتاح المكتب منتهي الصلاحية**. يوجد **شباك واحد** لاستلام مفتاح جديد (القفل):

- الأول يدخل الشباك ويستلم مفتاحاً جديداً ويعلّقه على اللوحة.
- الثاني كان ينتظر في الطابور. حين يصل الشباك **ينظر إلى اللوحة أولاً**، فيجد مفتاحاً صالحاً ويأخذه دون طلب جديد.
- الثالث مثله.

بدون النظرة الثانية للوحة سيطلب كل موظف مفتاحاً جديداً، أي 3 طلبات `login`. وبدون **الشباك** (القفل) قد يطلب الثلاثة في اللحظة نفسها.

```
@Synchronized                // the counter: one thread at a time
fun getValidToken(): String? {
    val token = storage.token
    if (token != null && !isExpired()) // second look at the key board
        return token
    return refresh()                // only if still expired
}
```

ج) Race Condition: ماذا لو لم نستخدم قفلاً؟

Thread	الخطوات	النتيجة
A	يقرأ الـ token فيجده منتهياً، فيبدأ <code>login</code>	يحفظ <code>tokenA</code>
B	يقرأ الـ token فيجده منتهياً، فيبدأ <code>login</code>	يحفظ <code>tokenB</code>
C	يقرأ الـ token فيجده منتهياً، فيبدأ <code>login</code>	يحفظ <code>tokenC</code>

النتيجة 3 طلبات `login`، والـ token المحفوظ في النهاية يعتمد على أيها انتهى أخيراً. هذا هو الـ Race Condition الذي يطلب المدير منعه.

د) لماذا عميل OkHttp؟ (منع الاستدعاء الذاتي)

✗ عميل OkHttp واحد للجميع:

1. `login()` يمر عبر `AuthInterceptor`.
2. الـ Interceptor يرى أن الـ token منتهٍ، فيستدعي `refresh()`.
3. `refresh()` يستدعي `login()` ... فنعود للخطوة 1 بلا نهاية ∞ (أو يتجمّد التطبيق على القفل: **deadlock**).

✓ عميلان منفصلان:

```
ProductApi -> okHttpWithAuth (has AuthInterceptor)
AuthApi    -> okHttpPlain (NO AuthInterceptor) -> login goes straight to the server
```

مع Hilt نميّز بين العميلين بـ **Qualifiers**، أي annotations مخصصة مثل `@AuthClient` و `@PlainClient`، ليعرف Hilt أي نسخة يعطي لكل Retrofit.

هـ) منع الحلقات اللانهائية (الحدود الثلاثة)

الحد	التنفيذ
login لا يمر بالـ <code>Interceptor</code>	عميل OkHttp منفصل (د)
محاولة تجديد واحدة لكل طلب	<code>refresh()</code> يُستدعى مرة واحدة، وإذا فشل يُرمى exception ولا يُعاد
Authenticator (عند 401) يعيد مرة واحدة	<code>if (response.priorResponse != null) return null</code> ، أي "سبق أن أعدت المحاولة، توقف"

و) Force Logout: كيف يصل الحدث من طبقة الشبكة إلى الشاشة؟

المشكلة: `AuthManager` يعمل في طبقة البيانات على thread الـ OkHttp، ولا يعرف شيئاً عن الشاشات، لكن يجب أن ينقل المستخدم لشاشة `Login`.

الحل: نمط **Observer** عبر `SharedFlow`:

#	طبقة البيانات: AuthManager	طبقة الواجهة: MainActivity
0	—	منذ onCreate تستمع: ... authManager.events.collect { ... }
1	التجديد فشل، فيستدعي clear()	—
2	← _events.tryEmit(ForceLogout) يُبث الحدث	—
3	—	يصل ForceLogout
4	—	popUpTo(inclusive) + navController.navigate(login) Toast("Session expired") +

- لماذا SharedFlow وليس StateFlow؟ لأن ForceLogout حدث لمرة واحدة. الـ StateFlow يحتفظ بآخر قيمة، فقد يُعاد تنفيذ الحدث عند تدوير الشاشة.
- نجمعه باستخدام repeatOnLifecycle(STARTED) حتى لا يعمل والتطبيق في الخلفية.

ز) التخزين المشفّر: ماذا يحدث فعلياً؟

```
"eyJhb..." --encrypt (AES key)--> "A9f$#k2..." --> saved to file
      ^
      |
key lives in Android Keystore
```

- الـ token يُشفّر بمفتاح AES قبل حفظه في الملف.
- المفتاح نفسه محفوظ في **Android Keystore**، داخل شريحة أمان أو منطقة معزولة في الجهاز، ولا يمكن نسخه خارجه.
- حتى لو سُرق ملف التطبيق، فهو مشفّر ولا يُفك إلا على نفس الجهاز وبنفس التطبيق.
- EncryptedSharedPreferences يقوم بكل هذا خلف واجهة SharedPreferences العادية.

ح) كيف يحسب التطبيق انتهاء الـ token؟

اللحظة	القيمة
عند الحفظ	tokenSavedAt = 1_700_000_000_000 (من System.currentTimeMillis())
عند الطلب	now = 1_700_000_061_000
العمر	now - tokenSavedAt = 61_000ms
القرار	61_000 ≥ 60_000 ، إذن منتهٍ

- نجعل الوقت يأتي من interface اسمه Clock. في التطبيق يُرجع الوقت الحقيقي، وفي الاختبار نستخدم FakeClock نقدّمه 61 ثانية فوراً دون انتظار.
- هامش أمان (اختياري): اعتبر الـ token منتهياً عند 55 ثانية بدل 60، حتى لا ينتهي أثناء سفر الطلب.

ط) كيف يعمل RecyclerView؟ (إعادة التدوير)

تشبيه: لديك 1000 منتج، والشاشة تتسع لـ 8 فقط. بدل صنع 1000 بطاقة، تصنع **نحو 10 بطاقات**. عند التمرير تخرج البطاقة من أعلى الشاشة، **فتمسح وتكتب عليها بيانات منتج جديد** وتدخل من الأسفل.

الجزء	دوره
RecyclerView	الحاوية التي تدير التمرير وإعادة التدوير
LayoutManager	ترتيب العناصر: عمودي (LinearLayoutManager) أو شبكي (GridLayoutManager)
ViewHolder	"البطاقة": يمسك مراجع عناصر الصف (صورة، عنوان، سعر)
Adapter.onCreateViewHolder	يصنع بطاقة جديدة، ويُستدعى قليلاً (نحو 10 مرات)
Adapter.onBindViewHolder	يكتب بيانات منتج على بطاقة موجودة، ويُستدعى كثيراً عند التمرير
ListAdapter + DiffUtil	عند وصول قائمة جديدة يحسب ما تغيّر فقط ويحدّثه مع حركة، بدل إعادة رسم الكل

ي) دورة MVVM كاملة في شاشة المنتجات

#	ماذا يحدث	الكود
1	الشاشة تبدأ الاستماع لحالة الـ ViewModel	<pre>← Fragment.onViewCreated viewModel.state.collect { render(it) }</pre>
2	الـ ViewModel يطلب البيانات عند إنشائه	<pre>viewModelScope.launch { ← init repository.getProducts() }</pre>
3	الحالة Loading، فيظهر ProgressBar	<pre>state = Loading</pre>
4	الطلب يمر بالـ Interceptor (يجدّ token إن لزم) ثم السيرفر	<pre>AuthInterceptor ← ProductApi ← Repository</pre>
5	نجاح: تُعرض القائمة. فشل: رسالة خطأ و زر Retry	<pre>errorText + / adapter.submitList(list) retryButton</pre>
6	عند تدوير الشاشة يُنشأ Fragment جديد مع نفس الـ ViewModel، فتظهر البيانات فوراً دون طلب جديد	—